基于最大流最小割的图像分割

目录

(一) 引言 1
(二) 基于最大流最小割的图像分割 1
2.1 Dinic 算法 1
2.2 最大流最小割与图像分割 2
(三) 总结 5
参考文献 6
附录 A：dinic.py 6
附录 B：main.py 6

《高级算法设计与分析》

22 级专硕 6 班

钟珊珊

22214624

2023 年 4 月 2 日

(一) 引言

图像分割是计算机视觉领域中的一个重要问题，其主要目的是将图像分成不同的区域，以便对每个区域进行更深入的分析和处理。在图像分割的应用中，基于图论中的最大流最小割定理是一种重要的算法，可以将图像分成不同的区域，例如将一张人脸照片分成头发、眼睛、鼻子等不同的部分，或将一张自然风景照片分成天空、山脉、水流等不同的部分，实现高效的图像分割。

基于图论中的最大流最小割定理指出，在一个网络图中，最大流等于最小割，即在网络图中找到最大的流量与最小的割点的问题等价。经典的最大流最小割算法有 Ford-Fulkerson 算法、Edmonds-Karp 算法、Dinic 算法等，应用领域非常广泛，例如图像分割、网络设计、社交网络分析等。在图像分割领域中，最大流最小割算法例如，在图像分割中，Ford-Fulkerson 算法可以将图像分成两个部分，从而实现图像二分割，而 Dinic 算法则可以实现更高效的图像分割，特别是在大规模图像分割中，其速度更快。

在本报告中，我们以 Dinic 算法为例，结合代码，对基于最大流最小割集的图像分割的实现思想进行了详细阐述和效果展示。

(二) 基于最大流最小割的图像分割

在本章节，我们结合代码对最大流最小割的图像分割建模进行阐述。我们首先介绍 Dinic 算法的建模和求解流程，最后结合代码实现，分析 Dinic 算法如何应用于图像分割。

2.1 Dinic 算法

Dinic 算法是一种用于解决最大流问题的增广路径算法。它建立在 BFS 算法的基础上，通过多次进行增广路径的寻找和更新，不断增加流量，直到无法找到增广路径为止。具体步骤如下（Python 实现见附录 dinic.py）[1]：

(1) 对原始流网络建立一个分层辅助网络，该网络表示每条边上还能流通的最大流量。

[1] 参考课程 PTT《第 2 讲 网络流算法》Page40-60

(2) 在分层辅助网络上执行 BFS 算法，从源点 s 开始寻找增广路径。每条边的容量为正值，但残量容量可能为 0 或负值，表示这条边已经满载或者流量方向与当前增广路径相反。
(3) 如果能够找到一条增广路径，就沿着路径更新残量容量，将沿路径的最小残量容量加入到当前流量中。同时，记录每个节点在增广路径中的前驱节点，以便后续重构增广路径。
(4) 如果无法找到增广路径，则算法结束，当前的流量即为最大流。
(5) 在分层辅助网络上多次执行步骤 2-4，直到无法找到增广路径为止。
(6) 对原始流网络中每条边的流量进行更新，得到最大流。

2.2 最大流最小割与图像分割

最大流最小割常用于解决计算机视觉中的能量方程最小化问题，所谓的能量方程通常是基于图结构得到的能量求解方法，这类能量方程可以普遍表示为

$$E(L) = \sum_{p \in P} D_p(L_p) + \sum_{(p,q) \in N} V_{p,q}(L_p, L_q)$$ [2]

其中 L 是图像 P 的一个标签结果；D_p为数据惩罚项，即像素点自身数据值（如：色彩、强度等）的能量评测项；$V_{p,q}$为数据互动项，考虑两两像素间的不连续关系（如空间或时间连续性）对整体能量值的影响。能量方程 E(L)就是计算在当前标签结果下的能量值。通常我们将能量方程最小化，从而求出可以使得总体能量最小的标签值情况 L，利用最终的像素分类结果得到图像分割等结果。

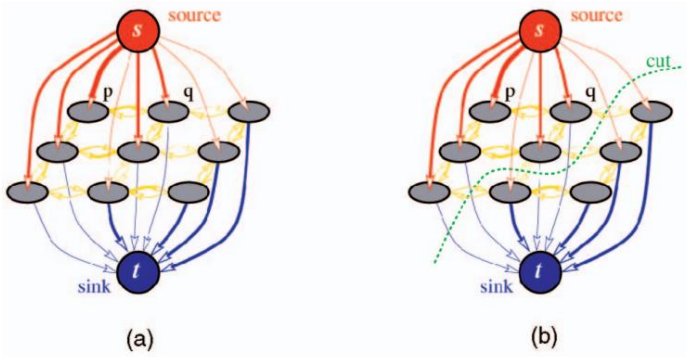


Fig. 2. Example of a directed capacitated graph. Edge costs are reflected by their thickness. A similar graph-cut construction was first used in vision by Greig et al. [15] for binary image restoration. (a) A graph $\mathcal{G}$. (b) A cut on $\mathcal{G}$.

图 1 基于最大流最小割的图像分割，右图中的 cut 把图分成了前景和背景，最后和 s 相连的边是前景，与 t 相连的是背景[3]

[2] 来源于参考文献[1]
[3] 来源于参考文献[1]

如图 1 基于最大流最小割的图像分割，右图中的 cut 把图分成了前景和背景，最后和 s 相连的边是前景，与 t 相连的是背景，在图像分割任务当中，利用最大流最小割的求解思路即把一张图像看成一个图结构，引入额外两个节点 s 和 t 来分别代表前景和背景，这样就将一个二元图像的分割问题转换成一个最大流最小割问题。

在图像分割问题图结构化的过程当中，一个很重要的问题是我们如何定义图结构当中边的权值，以利用最小化 E(L)的方式对问题求解。一般我们以 RGB 色彩接近程度衡量像素点是否属于同一类图像区域。因此在代码实现当中，我们对于终端节点 s、t 隐式边定义如下，即使用 RGB 的均值定义隐式边的权值：

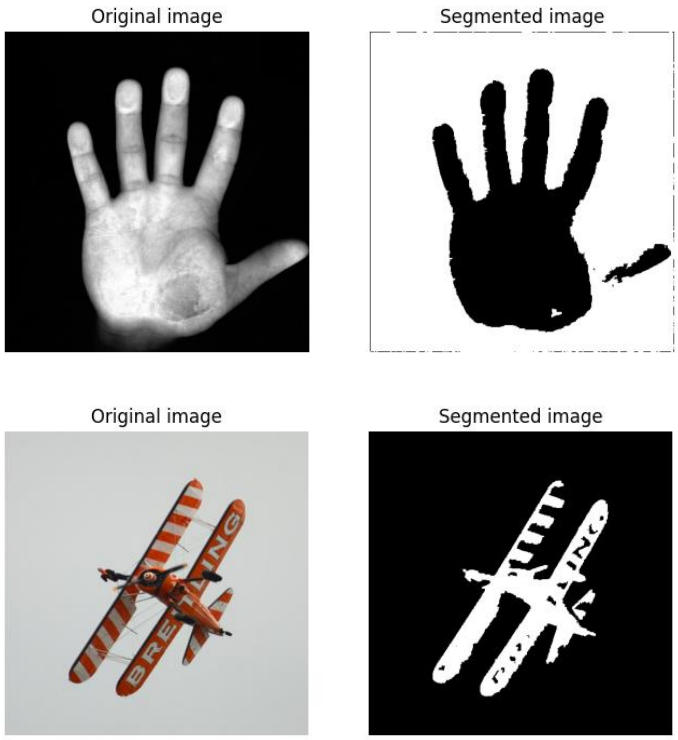
```
for y in range(height):
    for x in range(width):
        node_id = y * width + x
            if x == 0 or x == width - 1 or y == 0 or y == height - 1:
                s_cap.append(node_id)
                s_cap.append(0)
                t_cap.append(node_id)
                t_cap.append(0)
          else:
                b, g_, r = img[y, x]
                mean_rgb = (b+g_+r)/3
                s_cap.append(node_id)
                s_cap.append(b+g_+r)
                t_cap.append(node_id)
                t_cap.append(255 - mean_rgb)
```

此外，对于图中各个像素点之间的边，我们直接以相邻像素之间的像素值差的平方作为边的权值（用绝对值也是可以的）：

```
for y in range(height):
    for x in range(width):
        node_id = y * width + x
        if x < width - 1:
            right_node_id = str(y * width + x + 1)
            if node_dict.get(right_node_id, -1) == -1:
                node_dict[right_node_id] = [[], []]
            diff = np.sum((img[y, x] - img[y, x + 1]) ** 2)
            # diff = np.sum(np.abs(img[y, x] - img[y, x + 1]))
            node_dict[node_id][0].append(right_node_id)
            node_dict[node_id][1].append(diff)
```

```
        node_dict[right_node_id][0].append(node_id)
        node_dict[right_node_id][1].append(diff)
    if y < height - 1:
        bottom_node_id = str((y + 1) * width + x)
        if node_dict.get(bottom_node_id, -1) == -1:
            node_dict[bottom_node_id] = [[], []]
        diff = np.sum((img[y, x] - img[y + 1, x]) ** 2)
        # diff = np.sum(np.abs(img[y, x] - img[y + 1, x]))
        node_dict[node_id][0].append(bottom_node_id)
        node_dict[node_id][1].append(diff)
        node_dict[bottom_node_id][0].append(node_id)
        node_dict[bottom_node_id][1].append(diff)
```

基于上述设定，如图 2 基于最大流最小割的图像分割结果示例所示，我们可以获取到图像分割的二分结果。对于背景色彩较为的图片，例如手掌、飞机、披萨，基于最大流最小割的图像分割可以得到不错的分割效果。此外，算法对青蛙的轮廓也有较好的识别效果。但是，当背景颜色比较混杂或者背景颜色和目标物体的颜色接近时，算法的效果明显较差。例如飞机机翼中白色部分被识别成和天空，马与颜色较深的天空部分混杂在一起。为解决这一问题，需要对边的权值进行更细致的定义。



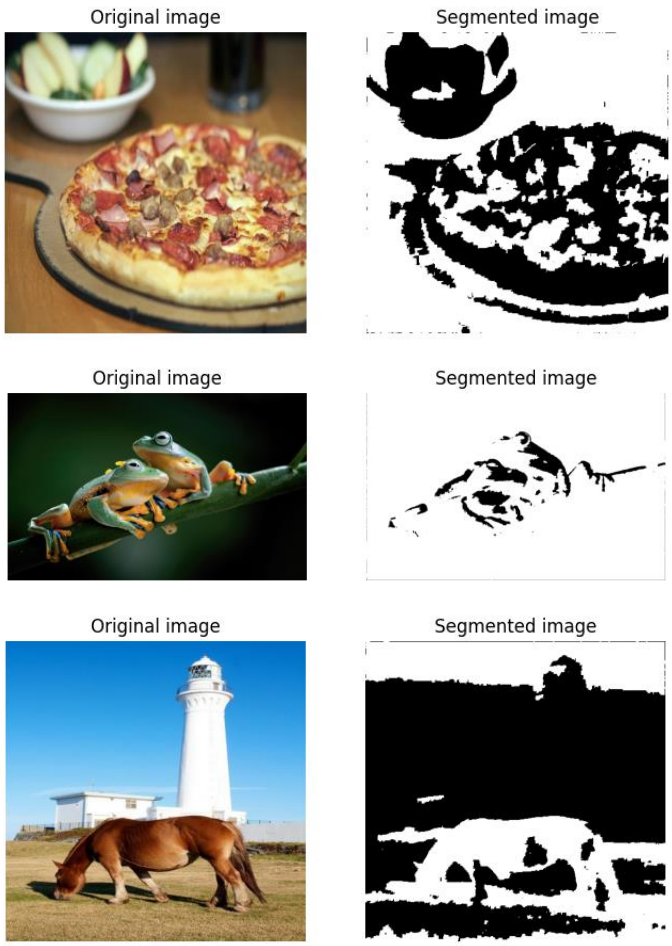


图 2 基于最大流最小割的图像分割结果示例

(三) 总结

在本报告中，我们详细介绍了基于最大流最小割的图像分割。具体而言，我们首先详细介绍了最大流最小割的求解算法 Dinic，并介绍了能量方程、最大流最小割如何应用在图像分割当中，对基于最大流最小割的图像分割效果进行了分析。基于最大流最小割的图像分割可以很好地区分背景和前景色彩区别大的图像，但是当背景和前景在色彩上区别不大时，需要进一步优化图结构的边权值设定。

除了最大流最小割思想的图像分割算法，还有许多其他的图像分割方法，例如基于边缘检测、基于区域生长、基于聚类等。每种方法都有其独特的特点和应用场景。近年来，随着深度学习技术的不断发展，基于深度学习的图像分割算法也成为了研究热点。例如，基于卷积神经网络的图像分割算法可以通过学习图像特征来实现更准确的分割结果。此外，还有一些基于生成对抗网络和图像语义分割的深度学习算法，也取得了很好的效果。

参考文献

[1] An Experimental Comparison of Min-Cut/Max-Flow Algorithms for Energy Minimization in Vision. Yuri Boykov and Vladimir Kolmogorov. In IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI), September 2004

附录 A：main.py

```
from dinic import *
import cv2
import numpy as np
import matplotlib.pyplot as plt
import warnings

warnings.filterwarnings('ignore')

def seg_maxflow(img):
    height, width = img.shape[:2]

    node_list = []
    s_cap = []
    t_cap = []
    node_dict = {}

    for y in range(height):
        for x in range(width):
            node_id = str(y * width + x)
            node_list.append(node_id)

            # 设置源点（前景 s）和汇点（背景 t）
            if x == 0 or x == width - 1 or y == 0 or y == height - 1:
                s_cap.append(node_id)
                s_cap.append(0)
                t_cap.append(node_id)
                t_cap.append(0)
```

```
else:
        b, g_, r = img[y, x]
        mean_rgb = (b+g_+r)/3
        s_cap.append(node_id)
        s_cap.append(b+g_+r)
        t_cap.append(node_id)
        t_cap.append(255 - mean_rgb)

# 设置结点之间的边
if node_dict.get(node_id, -1) == -1:
        node_dict[node_id] = [[], []]

if x < width - 1:
        right_node_id = str(y * width + x + 1)
        if node_dict.get(right_node_id, -1) == -1:
                node_dict[right_node_id] = [[], []]

        diff = np.sum((img[y, x] - img[y, x + 1]) ** 2)
        # diff = np.sum(np.abs(img[y, x] - img[y, x + 1]))
        node_dict[node_id][0].append(right_node_id)
        node_dict[node_id][1].append(diff)
        node_dict[right_node_id][0].append(node_id)
        node_dict[right_node_id][1].append(diff)
if y < height - 1:
        bottom_node_id = str((y + 1) * width + x)
        if node_dict.get(bottom_node_id, -1) == -1:
                node_dict[bottom_node_id] = [[], []]

        diff = np.sum((img[y, x] - img[y + 1, x]) ** 2)
        # diff = np.sum(np.abs(img[y, x] - img[y + 1, x]))
        node_dict[node_id][0].append(bottom_node_id)
        node_dict[node_id][1].append(diff)
        node_dict[bottom_node_id][0].append(node_id)
        node_dict[bottom_node_id][1].append(diff)
```

格式: [节点名, 后继节点的名称, 当前节点到各个后继的流量] （None 代表流量无穷大）

```
    graph = [
        ["s", node_list, s_cap],
        ["t", [], []],
    ]

    for k in node_dict.keys():
        next_node_list = node_dict[k][0]
        next_cap = node_dict[k][1]

        if k in node_list:
            next_node_list.append("t")
            next_cap.append(t_cap[node_list.index(k)])

        graph.append([k, next_node_list, next_cap])

    name_index_dict = dict()
    node_list = []
    for i in range(len(graph)):
        node_list.append(create_node(graph[i][0], graph[i][1], graph[i][2]))
        name_index_dict[graph[i][0]] = i

    # 调用算法求解最大流
    routes = Dinic_Solve("S", "E", node_list, name_index_dict)

    # 绘制图像分割示意图
    labels = np.zeros_like(img)
    for y in range(height):
        for x in range(width):
            node_id = str(y * width + x)
            if node_id in routes:
                labels[y, x] = [1, 1, 1]

    return labels


def plot(img, labels):
    # 显示原始图像和分割后的图像
```

```
    _, ax = plt.subplots(ncols=2, figsize=(8, 4), sharex=True, sharey=True)
    ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    ax[0].set_title('Original image')
    ax[1].imshow(labels, cmap='gray')
    ax[1].set_title('Segmented image')

    # 去除横纵坐标
    for a in ax:
        a.set_axis_off()

    plt.show()


if __name__ == '__main__':
    for i in range(5):
        img = cv2.imread(f'image/{i}.jpg')
        labels = seg_maxflow(img)
        plot(img, labels)
```

附录 B：dinic.py

```
import numpy as np


class Node:
    def __init__(self, name, arc_dict):
        self.name = name
        self.arc_dict = arc_dict


def create_node(name, next_list, flow_list):
    arc_dict = {}
    for i in range(len(next_list)):
        arc_dict[next_list[i]] = flow_list[i]
    return Node(name, arc_dict)
```

```
def create_level_graph(s, e, node_list, name_index_dict):
    level_graph = np.zeros((len(node_list), len(node_list))).tolist()
    cur_layer = [s]
    all_node = set()
    all_node.add(s)
    next_layer = set()
    while len(cur_layer) > 0:
        for node_name in cur_layer:
            node = node_list[name_index_dict[node_name]]
            for key in node.arc_dict.keys():
                if key not in all_node and (node.arc_dict[key] is None or
node.arc_dict[key] > 0):
                    level_graph[name_index_dict[node_name]
                                ][name_index_dict[key]] = node.arc_dict[key]
                    next_layer.add(key)
                    all_node.add(key)
        cur_layer = list(next_layer)
        next_layer = set()
    return level_graph if e in all_node else None


def Dinic_Solve(s, e, node_list, name_index_dict):
    routes = []
    s_index = name_index_dict[s]
    e_index = name_index_dict[e]
    level_graph = create_level_graph(s, e, node_list, name_index_dict)
    while level_graph is not None:
        res_list = []
        while True:
            res = bfs(e_index, [s_index], None, level_graph)
            if res is None:
                break
            # 更新 level graph
            route, flow = res
            for i in range(len(route) - 1):
                if level_graph[route[i]][route[i + 1]] is not None:
```

```
                level_graph[route[i]][route[i + 1]] -= flow
            # 追加记录增广路径
            res_list.append(res)
            routes.append([[node_list[n].name for n in res[0]], res[1]])
        # 更新残存网络
        for res in res_list:
            update(res, node_list)
        # 重新构造 level graph
        level_graph = create_level_graph(s, e, node_list, name_index_dict)
    return routes


def update(res, node_list):
    route, flow = res
    for i in range(len(route) - 1):
        n1 = node_list[route[i]]
        n2 = node_list[route[i + 1]]
        # 正向更新 n1 -> n2 剩余流量减少
        if n2.name in n1.arc_dict.keys() and n1.arc_dict[n2.name] is not None:
            n1.arc_dict[n2.name] = n1.arc_dict[n2.name] - flow
        # 反向更新 n2 -> n1 剩余流量增加
        if n1.name in n2.arc_dict.keys() and n2.arc_dict[n1.name] is not None:
            n2.arc_dict[n1.name] = n2.arc_dict[n1.name] + flow


def bfs(e_index, cur_route, last_flow, level_graph):
    if cur_route[-1] == e_index:
        return cur_route, last_flow
    for next_node in range(len(level_graph)):
        if next_node not in cur_route:
            if level_graph[cur_route[-1]][next_node] is None or
level_graph[cur_route[-1]][next_node] > 0:
                flow = min_flow(
                    level_graph[cur_route[-1]][next_node], last_flow)
                cur_route.append(next_node)
                res = bfs(e_index, cur_route, flow, level_graph)
                if res is not None:
```

```
                return res
            cur_route.pop(-1)


def min_flow(f1, f2):
    return f1 if f2 is None else (f2 if f1 is None else min(f1, f2))
```